COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

PACKAGE 07 / VOLUME 06

Native Integration Guide

Python kernels to C++ and platform systems

UPPER UNDERGRADUATE
Modules 26-30 · Lessons 26.1-30.4 · Gate F

Edition: September 28, 2026

Original expanded instruction, worked examples, quizzes, implementation contracts and evidence-based review. The learner performs the projects and platform/human validation. This self-study curriculum is not an accredited degree.

Contents

Native Integration Guide **4**

From reference kernels to engine subsystems . . . 4

Build the offline audio harness . . . 4

Recommended engine interfaces . . . 5

Physics transfer . . . 5

Animation and event transfer . . . 5

Audio transfer . . . 6

AI transfer . . . 6

Network and service transfer . . . 6

Packaging and save transfer . . . 7

Completion checklist . . . 7

Reference Directory **8**

S01: Collision documentation . . . 8

S02: Simulation documentation . . . 8

S03: glTF tutorial: Skins . . . 8

S04: glTF 2.0 specification . . . 8

S05: Writing a callback function . . . 8

S06: SDL_AudioStreamCallback . . . 8

S07: Xbox Accessibility Guidelines . . . 8

S08: CS188: Informed search . . . 9

S09: Introduction . . . 9

S10: CS188: Minimax 9

S11: Automatic differentiation with torch.autograd 9

S12: Networked movement in the Character Movement Component 9

S13: Snapshot Interpolation 9

S14: GGPO SDK repository 9

S15: Transaction documentation 10

S16: Atomic Commit in SQLite 10

S17: os.replace documentation 10

S18: zipfile documentation 10

S19: Fix Your Timestep! 10

S23: XAG 103: Additional channels for audio and visual cues 10

S24: Deterministic Lockstep 10

S25: XAG 107: Input 10

S26: XAG 101: Text display 11

Original program reading IDs 11

Native Integration Guide

From reference kernels to engine subsystems

The Python code in this package is intentionally small and inspectable. Each exercise implements one algorithmic contract with supplied tests. A real engine adds lifetime, ownership, thread scheduling, resource limits, platform behavior, diagnostics and complete subsystem state. Passing a Python kernel does not mean those integration concerns have been solved.

The C++ sample is a native offline audio harness, not a full game engine, audio library or production network stack. There is no bundled finished Gate F game. The learner's engine from Volume 5 remains the integration target.

Build the offline audio harness

```
cmake -S starter/cpp -B build/audio
cmake --build build/audio --config Release
ctest --test-dir build/audio -C Release --output-on-failure
```

For a direct GCC/Clang-style build:

```
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic \
  starter/cpp/audio_harness.cpp -o audio_harness
./audio_harness
```

Use your installed compiler and record its version. The package's actual execution environment is documented in evidence/TESTED_ENVIRONMENT.md. Portability intent is not a claim that every OS/compiler/device was tested.

The harness prepares a 2,048-entry sine table before rendering. It has eight voices, each gain bounded to 0.125 in magnitude, a 64-command usable queue and a maximum of 16 commands consumed per render block. The render format is fixed at 256 mono frames, 48 kHz. All these are teaching choices. A platform callback with another block size or sample format requires an adapter and fresh tests; do not pass a short buffer to a function that assumes 256 frames.

The single-producer/single-consumer queue uses one unused slot to distinguish full from empty. Producer writes a payload, then publishes the write index with release ordering; the consumer acquires that publication before reading. The consumer similarly releases slots through the read index. The design assumes exactly one producer and one consumer, and its index type must be always lock-free on the selected target. Neither assumption is validated by simply compiling on another platform.

The supplied test exercises ring empty/full/FIFO/wrap behavior on one thread, two identically driven mixers over 100 blocks, finite sample bounds and invalid command handling. It does not stress concurrent producer/consumer interleavings, open a device or measure hardware underruns. Add a stress test and platform callback before completing Lesson 27.3. Do not read the consumer-owned diagnostic concurrently without adding a valid synchronization or

snapshot mechanism.

Recommended engine interfaces

The following signatures are illustrative contracts, not ready-to-link implementations:

```
PhysicsStep(world, fixed_dt, input_snapshot) -> contacts, state
AnimationEvaluate(rig, state, sample_time) -> local_pose
AIUpdate(readonly_world, decision_budget) -> intentions, trace
AudioEnqueue(bounded_command) -> accepted_or_full
NetworkReceive(validated_envelope) -> bounded_message_queue
ApplyAuthoritativeSnapshot(snapshot, pending_inputs) -> corrected_state
SavePrepare(versioned_snapshot) -> validated_bytes
SaveActivate(staged_bytes) -> success_or_recoverable_error
```

Assign ownership before implementing them. Simulation owns authoritative transforms. Animation may propose motion, but the final owner validates and applies it once. AI actions own their outstanding requests and release them on cancellation. The audio consumer owns active voices and callback-visible data until a safe retirement point. Network parsing does not mutate arbitrary engine state directly; it produces validated messages consumed at a defined boundary.

Physics transfer

Move stable-ID scene data and collision queries into a fixed-step world. Avoid pointers in serialized/replay state. Decide when spawns/despawns become visible and how handles detect stale generations. Preserve candidate order when repeatability requires it.

Implement native finite/range checks before indexing or allocation. Python integers do not overflow like fixed-width native integers; resource products such as width*height*channels need checked arithmetic. Finite floating-point values can still exceed a useful simulation scale. Declare the supported world/object scale and reject out-of-domain data rather than advertising robustness for arbitrary magnitudes.

Run analytic and oracle tests in native code, not only against the Python reference. Add the maintained-library adapter as a separate comparison target with a pinned revision. Record differing conventions at the adapter boundary.

Animation and event transfer

Use strong naming or types for local/model/world spaces. Test mesh-node transforms and inverse bind data with a non-identity scene. Validate hierarchy cycles, joint indices and weight sums during asset processing; repeat critical trust-boundary checks when runtime data can change.

Give animation state, action state and emitted events separate identities. A visual blend does not authorize a gameplay attack. A canceled action must release movement/navigation ownership even when its pose continues fading out. Use generation-aware event IDs so a delayed marker from an old action cannot affect a new action with the same entity number.

Keep presentation deduplication outside restored rollback state, with a bounded retention window and an old-event policy. Record exactly which event consequences are reversible simulation updates, presentation effects or persistent external results.

Audio transfer

Choose a platform API such as SDL3 or PortAudio using its current official documentation [S05, S06]. Prepare conversion, buffers, voices and assets before starting the callback. Maintain phase across blocks and define mix headroom. Bound both active voices and commands processed per callback. A bounded queue alone does not bound a consumer loop that drains while a producer keeps replenishing it.

Implement device start/stop/loss and sample-format adaptation outside the callback where required. Record the API's actual callback frame sizes, errors/underrun telemetry and stress conditions. Use low gain for first audible tests. Avoid maximum-volume test tones. If the device path is unavailable, keep the required lab marked deferred and retain the offline evidence under its narrower label.

AI transfer

An AI budget should name the unit: expanded nodes, simulated states, queries, microseconds or a combination. Include preprocessing and cancellation overhead in the complete subsystem budget. Tag asynchronous navigation/model work with request and world generations; ignore stale replies.

Use immutable snapshots or another explicit concurrency model for observations. A trace should include the chosen action, rejected alternatives and invalid observations. Keep a bounded legal fallback available before expensive search or inference begins. Store the feature/model schema in the artifact and validate it at load time.

Network and service transfer

network_lab.py runs two simulated clients and one server in one process. Its event queue injects seeded loss, delay, duplication and reorder. It uses scalar motion and a contiguous input acknowledgement. Its last 80 ticks are a reliable drain used to test convergence; that is not a claim that a permanently partitioned connection eventually converges.

For the complete module, connect the same schemas to local sockets or separate processes, then add clock, buffer, disconnect and malformed-message tests. Sender identity in an in-process function call is synthetic; a real session/authentication adapter must establish a principal before ownership checks. Local teaching tests need no public service.

service_lab.py uses SQLite to atomically record a result and update a balance. It injects a pre-commit failure and a post-commit lost response, checks exact/conflicting retries and reads state in a fresh interpreter. It does not implement a full match registry, real identity provider, distributed database or concurrent-load test. Add those required local registry/ownership

behaviors under a clearly bounded scope.

Packaging and save transfer

release_lab.py uses temporary directories and exception injection to illustrate staged activation and save replacement. It calls file flush/fsync before replacement but does not implement a cross-platform directory-sync/durability contract. No power-cut test was performed. A production durability promise requires a platform-specific design and evidence [S16, S17].

Data-only mods still need independent caps on metadata, allocation and actual payload bytes. Archive path handling, symbolic links, compression limits and duplicate filenames are separate boundaries from JSON schema validation [S18]. Keep a clean-folder launch test that has no access to the author’s accidental working files.

Completion checklist

For every subsystem, produce an interface contract, independently checked example, invalid-input tests, lifetime/cancellation tests, measured work/storage limits and one integration failure capture. Run the native implementation through your existing build/test pipeline. Label local algorithm tests, actual device/platform runs, process transport tests and human observations separately.

Only the last category can establish that a reviewer completed the loop or understood a cue. A script cannot certify its own evidence fields. Use CAPSTONE_SPEC and the empty templates to collect real results on the learner’s project.

Reference Directory

S-prefixed entries are the focused primary-source companions consulted for this expansion, September 28, 2026. Numbers are stable identifiers; gaps are intentional. R-prefixed entries preserve the original program reading directory. External books, papers, documentation and library source are not bundled; the explanations, exercises and demonstration code in this package are original teaching material. Pin installed versions in your own evidence rather than treating a mutable documentation page as an installed build.

S01: Collision documentation

Box2D project. Shape queries, broad phase, dynamic tree and time-of-impact limitations.

Open primary source

S02: Simulation documentation

Box2D project. Stepping, contacts, solver settings and the documented determinism contract.

Open primary source

S03: glTF tutorial: Skins

Khronos Group. Joint transforms, inverse bind matrices and mesh-node correction.

Open primary source

S04: glTF 2.0 specification

Khronos Group. Normative asset, animation and skinning data contracts.

Open primary source

S05: Writing a callback function

PortAudio project. Callback constraints and real-time-safe execution discipline.

Open primary source

S06: SDL_AudioStreamCallback

SDL project. SDL3 audio stream callback contract; consult the API for the chosen integration.

Open primary source

S07: Xbox Accessibility Guidelines

Microsoft. Game accessibility design/testing guidance, not a legal-compliance certification.

Open primary source

S08: CS188: Informed search

University of California, Berkeley. A*, heuristics and search guarantees under stated assumptions.

Open primary source

S09: Introduction

BehaviorTree.CPP project. Behavior-tree concepts, statuses and implementation context.

Open primary source

S10: CS188: Minimax

University of California, Berkeley. Adversarial search and alpha-beta reasoning.

Open primary source

S11: Automatic differentiation with torch.autograd

PyTorch project. Gradient/parameter concepts; framework use is optional for the core synthetic exercise.

Open primary source

S12: Networked movement in the Character Movement Component

Epic Games. A maintained engine comparison for authority, prediction and correction.

Open primary source

S13: Snapshot Interpolation

Glenn Fiedler. Author’s technical explanation of interpolation, jitter and visual delay.

Open primary source

S14: GGPO SDK repository

GGPO authors. Primary implementation/documentation for rollback networking concepts.

Open primary source

S15: Transaction documentation

SQLite project. Transaction boundaries and behavior.

Open primary source

S16: Atomic Commit in SQLite

SQLite project. Atomicity assumptions and storage failure considerations.

Open primary source

S17: os.replace documentation

Python Software Foundation. Replacement semantics and platform/filesystem conditions.

Open primary source

S18: zipfile documentation

Python Software Foundation. Archive processing and extraction boundaries.

Open primary source

S19: Fix Your Timestep!

Glenn Fiedler. Simulation/render time separation and work-budget hazards.

Open primary source

S23: XAG 103: Additional channels for audio and visual cues

Microsoft. Critical information through more than one channel.

Open primary source

S24: Deterministic Lockstep

Glenn Fiedler. Input synchronization and deterministic simulation requirements.

Open primary source

S25: XAG 107: Input

Microsoft. Input/remapping accessibility considerations.

Open primary source

S26: XAG 101: Text display

Microsoft. Readable interface text and testing considerations.

Open primary source

Original program reading IDs

These inherited assignments remain part of the full study route. S entries above focus the expanded lesson tasks; they do not replace all external study.

R17: RFC 9293: Transmission Control Protocol — IETF / RFC Editor. Transport semantics; reference reading after networking foundations. Assigned source

R19: 15-462/662 Computer Graphics — Carnegie Mellon University. Rendering, geometry and animation course structure and exercises. Assigned source

R20: 15-466/666 Computer Game Programming — Carnegie Mellon University. Runtime systems and game-production programming. Assigned source

R22: SDL3 documentation — SDL project. Windowing, input, audio and platform services; do not mix SDL2 and SDL3 examples. Assigned source

R23: Physically Based Rendering, fourth edition — Matt Pharr, Wenzel Jakob, Greg Humphreys. Rendering mathematics and implementation; use chapter titles in lesson assignments. Assigned source

R26: Programming with C++ in Unreal Engine — Epic Games. Engine comparison after C++ foundations, not a beginner substitute. Assigned source

R28: Box2D documentation — Box2D project. Collision, constraints and simulation reference. Assigned source

R29: Xbox Accessibility Guidelines — Microsoft. Accessibility design and testing; not a legal-compliance certificate. Assigned source

R45: AddressSanitizer documentation — LLVM Project. Memory-error instrumentation, limitations and supported toolchains. Assigned source

R46: CS188 Introduction to Artificial Intelligence textbook — University of California, Berkeley. Search, games, decision processes, learning and probabilistic reasoning. Assigned source

R47: Dive into Deep Learning — Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola. Learning fundamentals, gradients, generalization and reproducible exercises; pin compatible framework versions. Assigned source